



Universidad Don Bosco
FACULTAD DE INGENIERÍA
ESCUELA DE COMPUTACIÓN
Desarrollo de aplicaciones con Software Propietario
Guía 3: Estructuras Repetitivas



I. Objetivos

Que el estudiante sea capaz de:

1. Declarar variables de acuerdo a los requerimientos de datos a manejar.
2. Aplicar la sintaxis de escritura de identificadores y comandos en C#.NET.
3. Definir todo tipo de expresiones complejas (matemáticas y lógicas) en un programa C#.NET.
4. Implementar las instrucciones de control repetitiva (For, While, Do-While) en el código fuente.
5. Diseñar la combinación apropiada de las diferentes estructuras vistas en el diseño de un programa cualquiera

II. Introducción Teórica

Declaración de variables de acuerdo a los requerimientos de datos a manejar:

Es muy importante elegir bien el tipo de dato a la hora de declarar una variable, esto nos ayudará a consumir menos recursos de memoria y lograr mayor eficiencia en nuestras aplicaciones.

Como hemos visto en guías anteriores, cada tipo de dato ocupa cierto espacio en memoria. Por lo tanto, debemos ser cuidadosos al momento de elegir un tipo de dato para almacenar la información, ya que de eso dependerá también la cantidad de memoria que usaran finalmente nuestras programas o aplicaciones.

Por ejemplo, cuando necesitemos almacenar datos que contienen solo números, como por ejemplo un número de teléfono, un número de DUI o ISSS, es mejor declararla de tipo String en vez de tipo Int o Short, dado que al declararlos como String, estos ocuparán en memoria solo la cantidad de bytes necesarios para representar a cada uno de los caracteres que compone la información, puesto que, no se harán cálculos sobre ellos.

Pero entonces cuando debemos declarar variables de tipo numérico, sencillo lo haremos solo cuando necesitemos almacenar datos con los cuales haremos operaciones o cálculos matemáticos con dichas variables.

También es necesario al elegir un tipo de dato numérico, tener en cuenta la cantidad de números que va a almacenar y si estos son de coma flotante o no. Debemos de tomar muy en cuenta los rangos de valores que puede tomar cada tipo de dato para hacer la elección correcta.

Por ejemplo, si sabemos que un número no pasara de 100, podríamos utilizar un Short en lugar de un Int o un Double, ahorrando espacio de memoria.

Estructuras Repetitivas

Permiten ejecutar un conjunto de sentencias repetidamente una cierta cantidad de veces o hasta que se cumpla una determinada condición. Al conjunto de sentencias se le denomina bucle y a cada repetición del cuerpo del bucle se denomina iteración.

Visual C#.Net cuenta con varias instrucciones para el control de repeticiones de bucles, veamos detalladamente cada una de ellas.

Ciclos While y Do-While el bucle se repetirá hasta que la condición se falsa.

Si usted utiliza la opción 1 (While), primero se va a evaluar la condición y, depende de la instrucción que usted utilice, se ejecutará el bucle si la condición es verdadera. Mientras que con la opción 2 (Do-While), el bucle se ejecutará por lo menos una vez y posteriormente, se ejecutará el bucle si la condición es verdadera.

Instrucciones While ... Do While

Sintaxis:

While (condición){ Instrucción(es); }	Do{ Instrucción(es); }while(condición)
---	--

Instrucción For

La instrucción For, se utiliza para repetir un conjunto de instrucciones, un número dado de veces.

Sintaxis:

For (Valor inicial; Condición; Incremento){ (Instrucciones) }

Donde:

- Valor Inicial: Es el valor inicial del contador.
- Valor Final: Es el valor final del contador.
- Condición: Puede ser cualquier condición que determine donde finalizara las iteraciones.
- Instrucciones: Son las instrucciones que se repetirán un número de veces dada.

III. Requerimientos

Nº	Cantidad	Descripción
1	1	PC con Microsoft Visual Studio 2026 .NET
2	1	Guía 3
3	1	Memoria USB

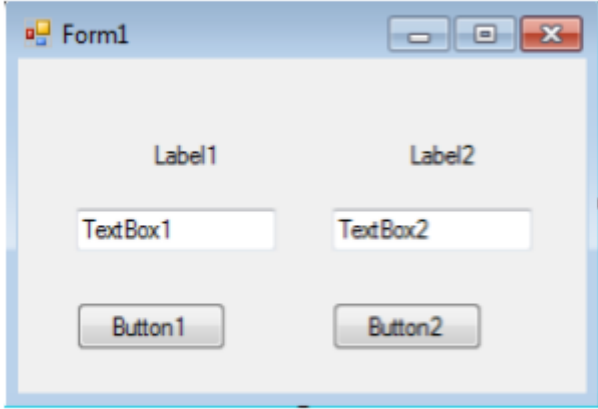
IV. Procedimiento

Para esta práctica deberá crear una carpeta con el nombre de “Práctica3DSP”, en la cual va a guardar todos los archivos de esta práctica.

Parte 1. Aplicación Utilizando FOR

Problema a Solucionar: “Calcular y mostrar el factorial de un número”

1. Cree un nuevo proyecto de Windows en C#.Net.
2. Diseñe el siguiente esquema de objetos (Figura 1.1) y asigne los valores a las propiedades que se muestran.



Propiedades			
Controles	Name	Text	Enable
Form1	frmFactorial	Calculando el factorial de un número	
Label1	lblNumero	Número:	
Label2	lblFactorial	Factorial:	
Textbox1	txtNumero	(Vacio)	
Textbox2	txtFactorial	(Vacio)	False
Button1	btnCalcular	&Calcular	
Button2	btnSalir	&Salir	

Nota: Puede cambiar las propiedades de formato de texto y el fondo del formulario.

3. Digite el código detallado a continuación, en el evento y objeto especificado.

Objeto	Evento
Form1 (frmFactorial)	(Declaraciones a nivel de clase)
btnCalcular	Click
<pre>private void btnCalcular_Click(object sender, EventArgs e) { int numero; long factorial = 1; if (txtNumero.Text == "") { MessageBox.Show("Ingrese un número"); txtNumero.Focus(); return; }</pre>	

```

}

numero =
Convert.ToInt32(txtNumero.Text);

for (int i = 1; i <= numero;
i++)
{
factorial *= i;
}

txtFactorial.Text =
factorial.ToString();
}

```

txtNumero

KeyPress

```

private void txtNumero_KeyPress(object sender, KeyPressEventArgs e)
{
if (!char.IsDigit(e.KeyChar) &&
!char.IsControl(e.KeyChar))
{
e.Handled = true;
}
}

```

btnSalir

Click

Redacte el código para salir!!

4. Guarde los cambios en el proyecto actual y realice pruebas con varios números. Responda estas interrogantes: ¿Cree que los tipos de variables elegidas son las correctas? Si es así agregue las que considere necesarias.

Parte 2. Aplicación Utilizando FOR-EACH

Problema a solucionar: “Crear una aplicación para encriptar y desencriptar un mensaje utilizando el código de César. Este código consiste en sustituir cada letra por la que se encuentra alfabéticamente n posiciones más adelante. A esta n se le conoce como llave”

1. Diseñe el siguiente esquema de objetos (ver Figura 1.2) y asigne los valores a las propiedades descritas.

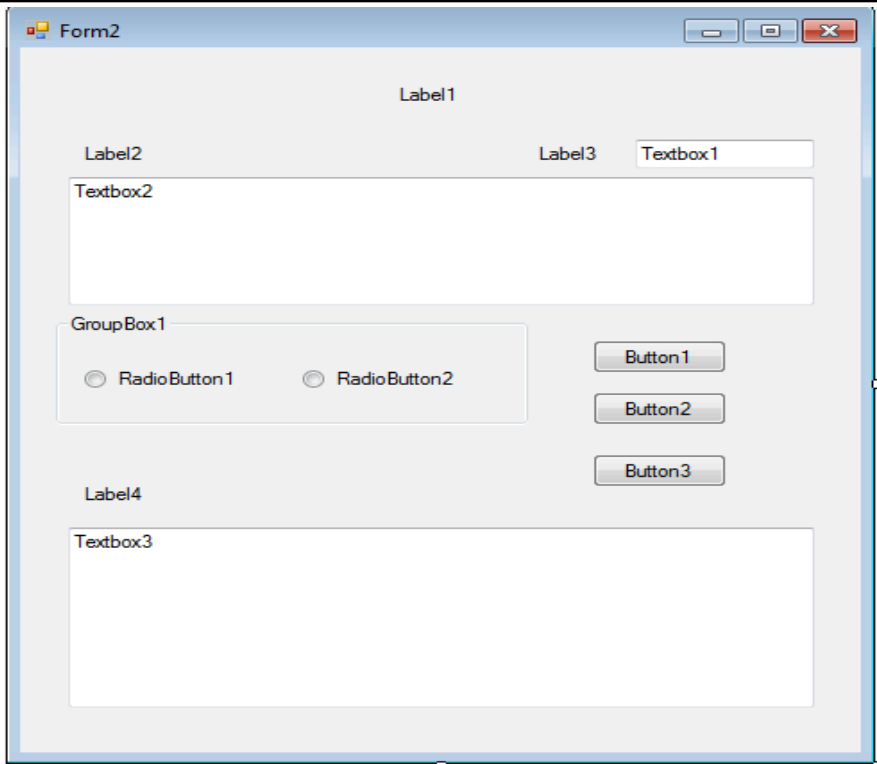


Figura 1.2 Diseño de objetos del Form2

Propiedades					
Controles	Name	Text	Enable	Checked	Multiline
Form2	frmCesar	Encriptador/Desencriptador de Código de César			
Label1	lbltitulo	Enciptamiento por código César			
Label2	lbltexto	Ingresa el texto a Encriptar/Desencriptar			
Label3	lblllave	Llave (Digite un número entero)			
Label4	lblresultado	(Vacio)			
Textbox1	txtLlave	(Vacio)			
Textbox2	txtTexto	(Vacio)			True
Textbox3	txtResultado	(Vacio)			True
GroupBox1	grOpcion	Seleccione una opción			
RadioButton1	rdbEnciptar	Enciptar		True	
RadioButton2	rdbDesenciptar	Desenciptar			
Button1	btnEnciptar	&Enciptar			
Button2	btnLimpiar	&Limpiar			
Button3	btnSalir	&Salir			

2. Digite el código detallado a continuación, en el evento y objeto especificado.

Objeto	Evento
Form1 (frmFactorial)	(Declaraciones a nivel de clase)
<pre>int llave;</pre>	
btnEncriptar	Click
<pre>private void btnEncriptar_Click(object sender, EventArgs e) { string texto = txtTexto.Text; string resultado = ""; llave = Convert.ToInt32(txtLlave.Text); if (rdbDesencriptar.Checked) { llave = -llave; } foreach (char letra in texto) { resultado += Convert.ToChar(letra + llave); } txtResultado.Text = resultado; }</pre>	
txtLlave	KeyPress
<pre>private void txtLlave_KeyPress(object sender, KeyPressEventArgs e) { if (!char.IsDigit(e.KeyChar) && !char.IsControl(e.KeyChar)) { e.Handled = true; } }</pre>	
rdbEncriptar	CheckedChanged
<pre>private void rdbEncriptar_CheckedChanged(object sender, EventArgs e) { btnEncriptar.Text = "Encriptar"; }</pre>	
rdbDesencriptar	CheckedChanged
<pre>private void rdbDesencriptar_CheckedChanged(object sender, EventArgs e) { btnEncriptar.Text = "Desencriptar"; }</pre>	

btnLimpiar	Click
<pre>private void btnLimpiar_Click(object sender, EventArgs e) { txtTexto.Clear(); txtLlave.Clear(); txtResultado.Clear(); rdbEncriptar.Checked = true; txtTexto.Focus(); }</pre>	
btnSalir	Click
Redacte el código para salir!!	

3. Guarde los cambios en el proyecto actual y realice pruebas con varios textos y llaves.

PARTE 3: APLICACIÓN QUE UTILIZA DO-WHILE

Problema a solucionar: “Crear una aplicación que dado un número lo muestre al revés.”

1. Agregue un nuevo formulario al proyecto y elíjalo como formulario de inicio.
2. Diseñe el siguiente esquema de objetos (Figura 1.3) y asigne los valores a las propiedades descritas.

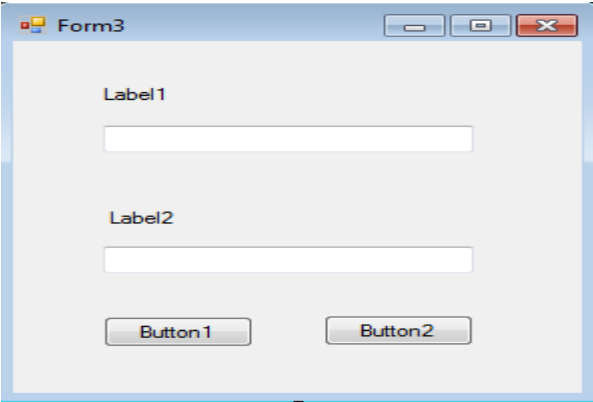


Figura 1.3 Diseño de objetos del Form3

Propiedades			
Controles	Name	Text	Enable
Form3	frmReves	Mostrar un número al revés	
Label1	lblNumero	Numero:	
Label2	lblReves	Al revés:	
Textbox1	txtNumero	(Vacio)	
Textbox2	txtReves	(Vacio)	False
Button1	btnCalcular	Calcular	
Button2	btnLimpiar	Limpiar	

3. Digite el código detallado a continuación, en el evento y objeto especificado.

Objeto	Evento
--------	--------

Form1 (frmReves)	(Declaraciones a nivel de clase)
<pre>public static bool IsNumeric(string dato) { int numero; return int.TryParse(dato, out numero); }</pre>	
btnCalcular	Click
<pre>private void btnCalcular_Click(object sender, EventArgs e) { if (!IsNumeric(txtNumero.Text)) { MessageBox.Show("Ingrese un número válido"); txtNumero.Focus(); return; } int numero = Convert.ToInt32(txtNumero.Text); int reves = 0; int digito; do { digito = numero % 10; reves = (reves * 10) + digito; numero = numero / 10; } while (numero > 0); txtReves.Text = reves.ToString(); }</pre>	
txtNumero	KeyPress
<pre>private void txtNumero_KeyPress(object sender, KeyPressEventArgs e) { if (!char.IsDigit(e.KeyChar) && !char.IsControl(e.KeyChar)) { e.Handled = true; } }</pre>	
btnLimpiar	Click
Hacer que caja txtNumero y txtReves se limpien	
Form3(frmReves)	(Declaraciones globales)
Crear función que valide que dato sea un número recibido llamada IsNumeric	

- Agregue el botón Salir.
- Guarde los cambios en el proyecto actual y realice pruebas con varios números.

PARTE 4: APLICACIÓN UTILIZANDO WHILE

Problema a solucionar: “Crear una aplicación que muestre la sucesión de ULAM dado un número entero positivo. La conjetura de ULAM comienza con cualquier numero positivo, Si es par se divide entre 2, si es impar, se multiplica por 3 y se le agrega 1, se obtienen enteros sucesivamente hasta obtener un 1”.

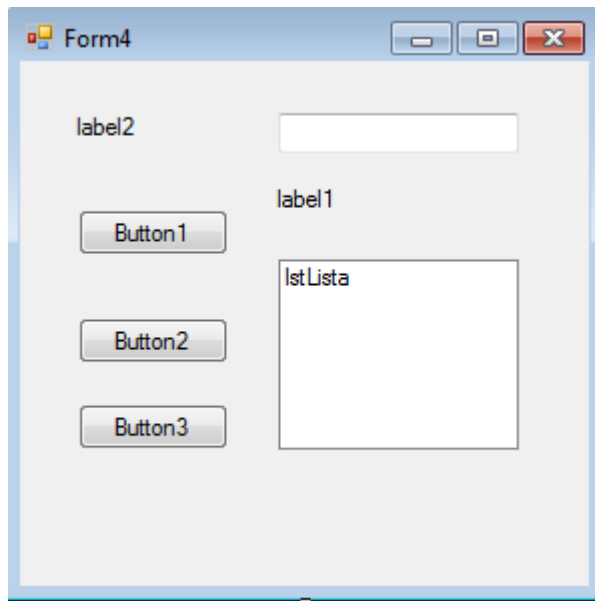


Figura 1.4: Diseño de objetos del Form4

Propiedades		
Controles	Name	Text
Form4	frmULAM	Sucesión de ULAM
Label1	lblSalida	Sucesión de ULAM:
ListBox1	lstLista	
Button1	btnCalcular	&Calcular
Button2	btnLimpiar	&Limpiar
Button3	btnSalir	&Salir
Label2	lblNumero	Ingrese un número positivo:
textBox1	txtnumero	

1. Agregue un nuevo formulario al proyecto y elíjalo como formulario de inicio.
2. Diseñe el siguiente esquema de objetos (Figura 1.4) y asigne los valores a las propiedades ahí descritas.
3. Proceda al código detallado a continuación, en el evento y objeto especificado.

Objeto	Evento
btnCalcular	Click
Redacte código para calcular Sucesión ULAM utilización while.	
btnLimpiar	Click
Redacte código para limpiar lista.	
btnSalir	Click
Redacte el código para salir	

4. Guarde los cambios en el proyecto actual y realice pruebas con varios números.
5. Valide la entrada de datos, cuando el usuario deja vacío el Input Box.

V. Ejercicios Complementarios

El instructor evaluará el desempeño de su trabajo dentro la clase. Estos ejercicios se evaluarán en la misma hora de clase.

Elabore un proyecto en el cual presente las soluciones de los siguientes problemas, cada uno en un formulario diferente.

1. Elabore una aplicación que muestre en un control ListBox a los números de la serie Fibonacci (0, 1, 1, 2, 3, 5, 8, 13, 21...) generados hasta un número introducido por el usuario.

Por Ejemplo: Si el usuario introduce valor 14, la serie a mostrar constaría de estos valores: 0, 1, 1, 2, 3, 5, 8 y 13. No se mostraría el 14 por ser mayor que el valor límite ingresado por usuario.

2. Solicite una serie de 4 números cualquiera, para luego calcular y mostrar el valor promedio de la lista y la desviación típica de toda la serie.

La desviación típica (σ) de una serie de N valores se calcula con la siguiente fórmula:

$$\sigma = \sqrt{\frac{\sum_{i=1}^N (X_i - \bar{X})^2}{N}} = \sqrt{\frac{(X_1 - \bar{X})^2 + (X_2 - \bar{X})^2 + \dots + (X_N - \bar{X})^2}{N}}$$

Por ejemplo: Si el usuario brinda los números 5, 6, 3, 4, el promedio $\bar{X}$ es de 4.5 y la desviación típica σ es de 1.118. Desarrolle una tabla de entradas-salidas que contenga un total de 3 pruebas extras.

VI. Análisis de resultados

Para la próxima semana:

1. Se presentan a continuación los enlaces de unos videos de youtube, que tratan el tema de los ciclos repetitivos For, While y Do While. Deberá de realizar los ejercicios mostrados en los videos como parte de la práctica de laboratorio.
 - <https://www.youtube.com/watch?v=jhagmT2glaU> (Ciclo For)
 - <https://www.youtube.com/watch?v=IMIm4cl0h18> (Ciclo While)
 - <https://www.youtube.com/watch?v=BjETdFiaj1c> (Ciclo Do While)

V. Bibliografía

- Título: Aprenda ya Microsoft Visual C#.NET. Autores: Sharp, John Jagger, Jon Coautor Publisher: Madrid, España: McGraw-Hill, 2002. Clasificación: Libro 005.362 S581 2002
- Título: C # Manual de Programación. Autores: Luis Joyanes Aguilar y Matilde Fernández Azuela Publisher: Madrid, España: McGraw-Hill, 2002 Clasificación: Libro 005.362 J88 2002